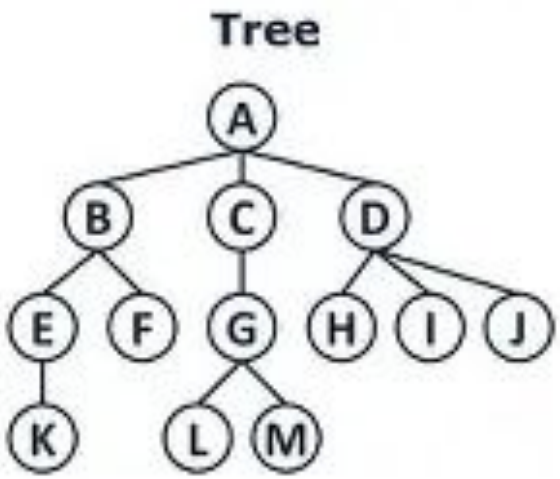
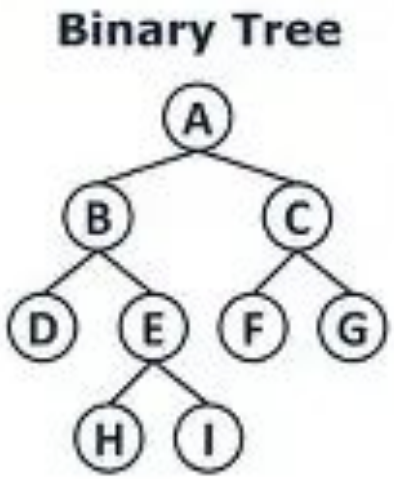
# BINARY TREE

By: Guillermo Andres De Mendoza Corrales

# Theory

A **binary tree** is a hierarchical data structure where each node has at most two children, referred to as the **left child** and **right child**. It is widely used in computer science for efficient data storage, retrieval, and manipulation. Each node in a binary tree consists of three components: the data, a pointer to the left child, and a pointer to the right child.



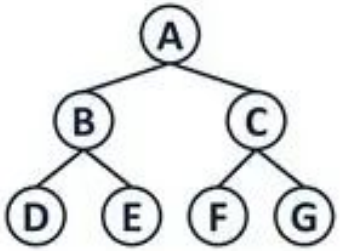
# Types

**1. Full Binary Tree**: Every node has either 0 or 2 children.

**2. Perfect Binary Tree**: All internal nodes have two children, and all leaf nodes are at the same level.

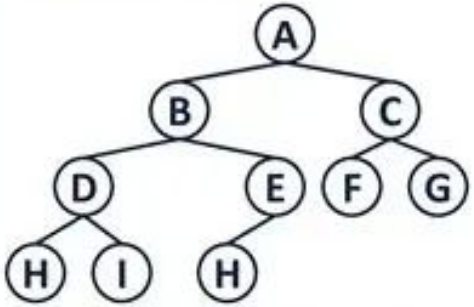
**3. Complete Binary Tree**: All levels are completely filled except possibly the last, which is filled from left to right.

**Perfect Binary Tree**




All Perfect Binary Trees are Complete Binary Trees, but reverse is not true.

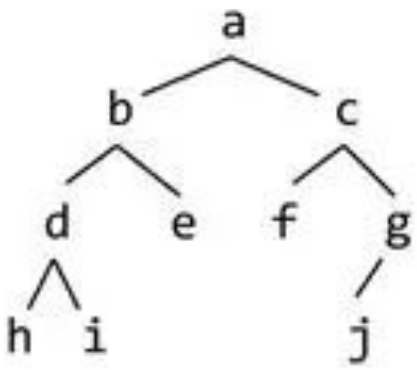
**Complete Binary Tree**



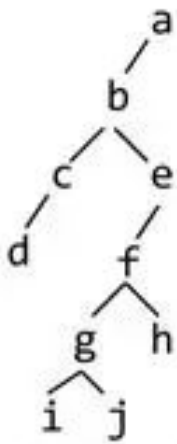

It is Complete Binary Tree, but not a Perfect Binary Tree.

# Types

**4. Balanced Binary Tree**: The height difference between the left and right subtrees of any node is at most 1.




A balanced binary tree

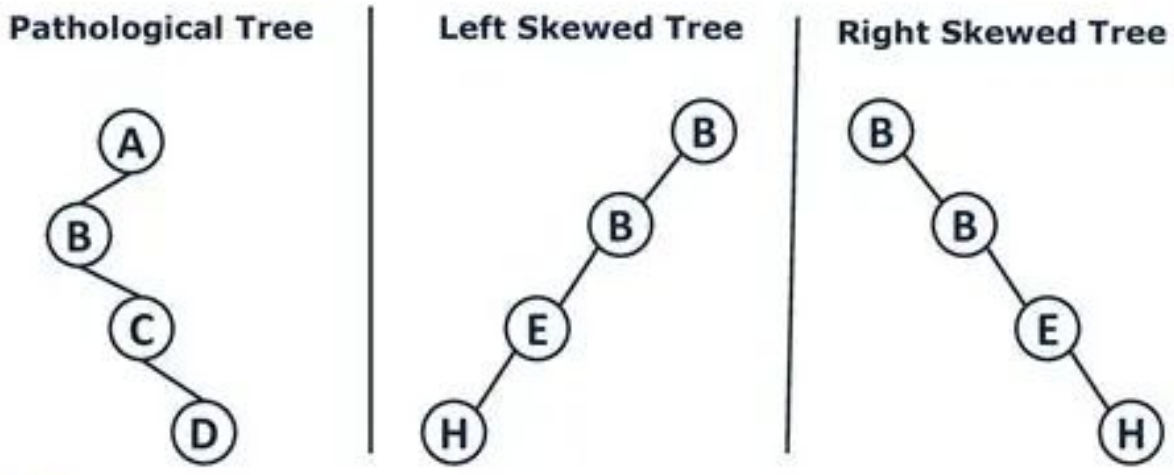



An unbalanced binary tree

# Types

**5. Skewed Binary Tree**: All nodes are aligned to either the left or right, forming a linear structure.

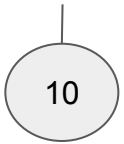
**6. Degenerate Tree**: Each parent node has only one child, making it similar to a linked list.



# Set

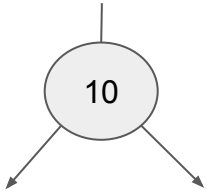
We are looking to save the information in a binary tree, following the rules:

1. Adding an new node if the Tree is empty:
   - always put it in the root of the three



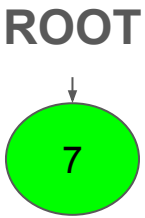

2. Adding an new node for a tree containing nodes:

- if the **new node** value is **smaller** than the **actual node** value, try to add the new node to the left of the current node
- if the **new node** value is **greater** than the **actual node**, value try to add the new node to the right of the current node
- if the new value is equals to the current node, its not possible to add the new node



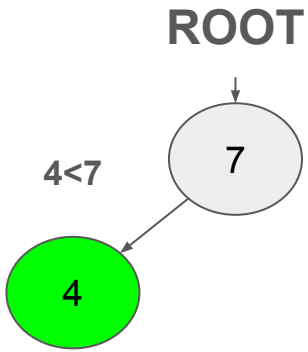
# Set example (add 7)

- Lest add this elements in order to an empty binary tree: [ **7, 4, 10, 3, 5,  12, 6, 10** ]



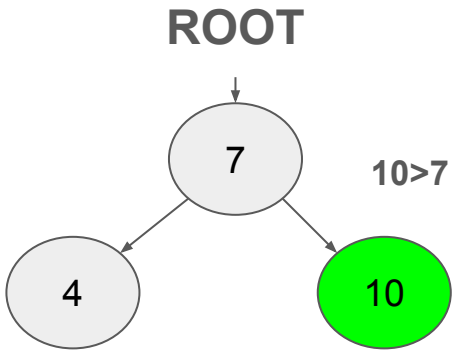
# Set example (add 4)

- Lest add this elements in order to an empty binary tree: [ **7, 4, 10, 3, 5,  12, 6, 10** ]



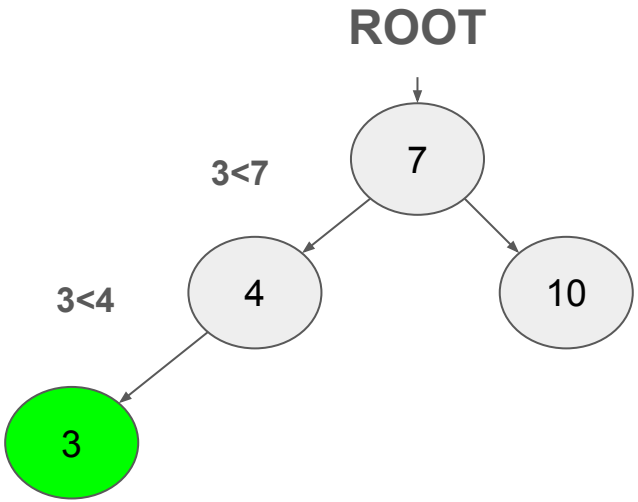
# Set example (add 10)

- Lest add this elements in order to an empty binary tree: [ **7, 4, 10, 3, 5, 12, 6, 10** ]



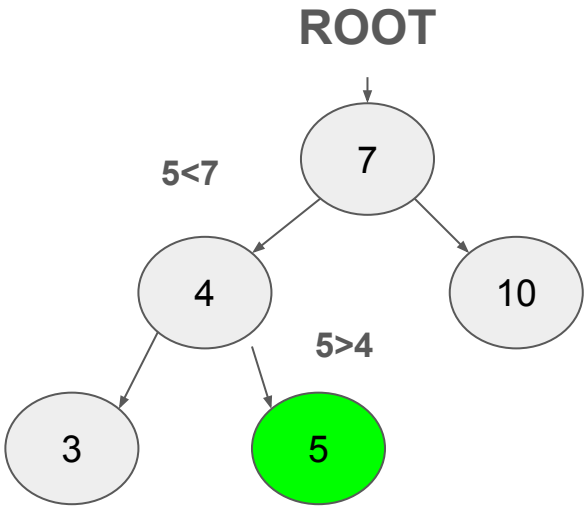
# Set example (add 3)

- Lest add this elements in order to an empty binary tree: [ **7, 4, 10, 3, 5,  12, 6, 10** ]



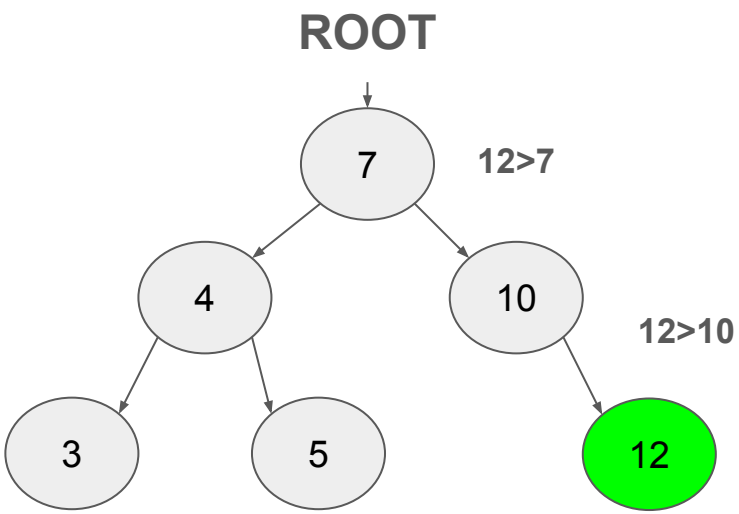
# Set example (add 5)

- Lest add this elements in order to an empty binary tree: [ **7, 4, 10, 3, 5, 12, 6, 10** ]



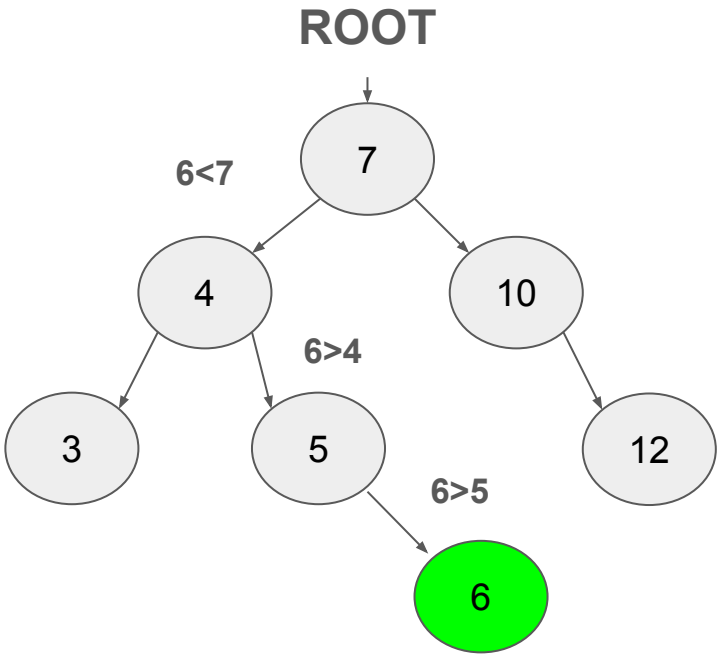
# Set example (add 12)

- Lest add this elements in order to an empty binary tree: [ **7, 4, 10, 3, 5, 12, 6, 10** ]



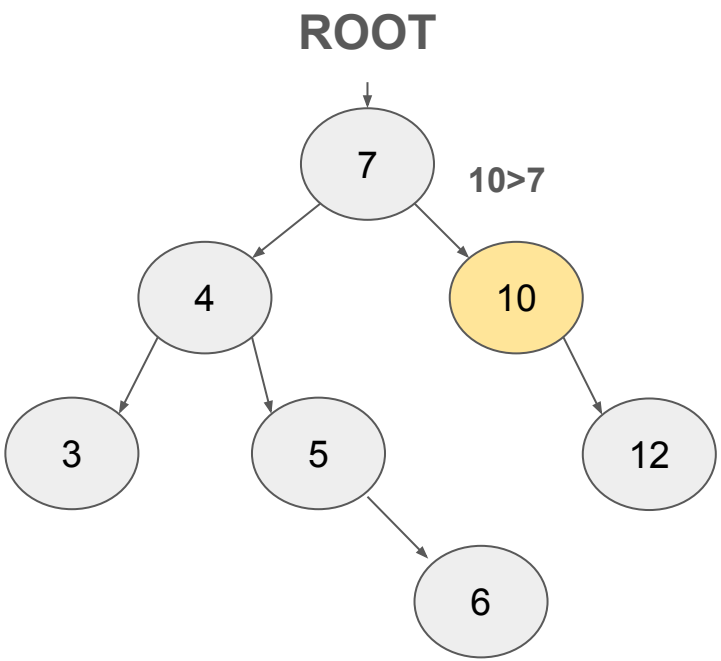
# Set example (add 6)

- Lest add this elements in order to an empty binary tree: [ **7, 4, 10, 3, 5, 12, 6, 10** ]

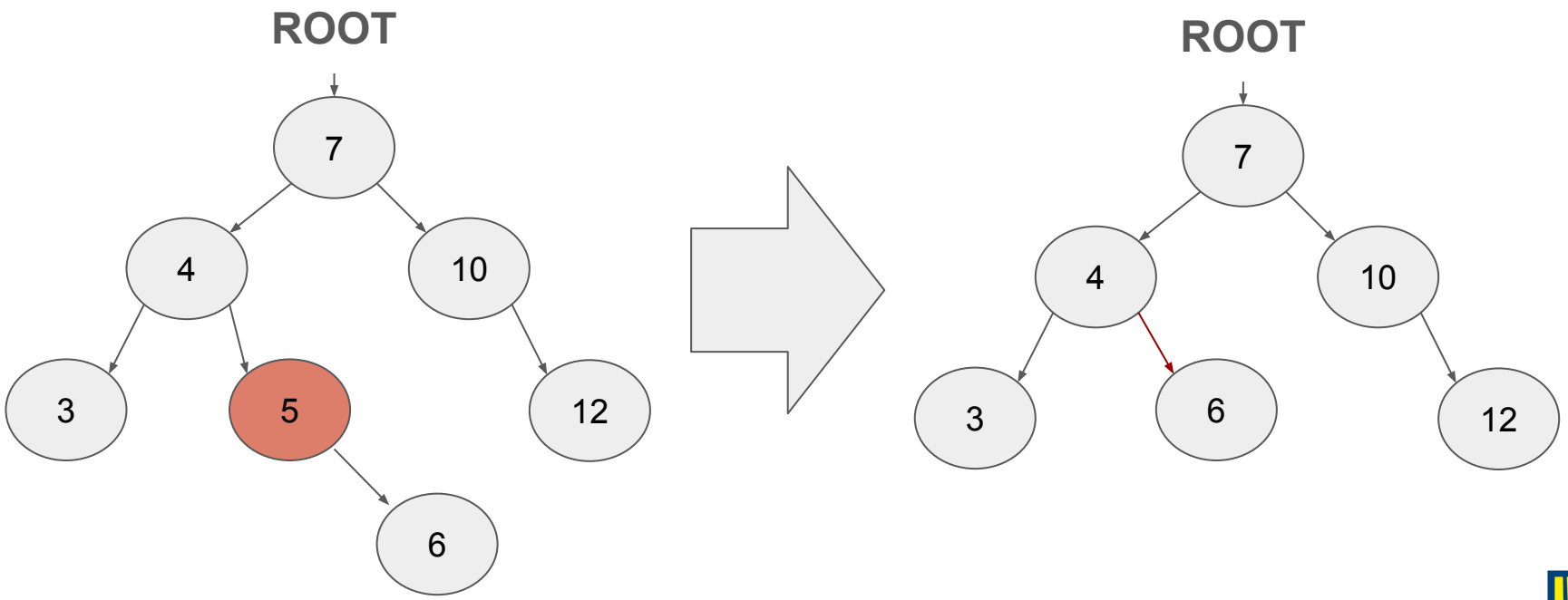


# Set example (add 10) - Repeated

- Lest add this elements in order to an empty binary tree: [ **7, 4, 10, 3, 5, 12, 6, 10** ]

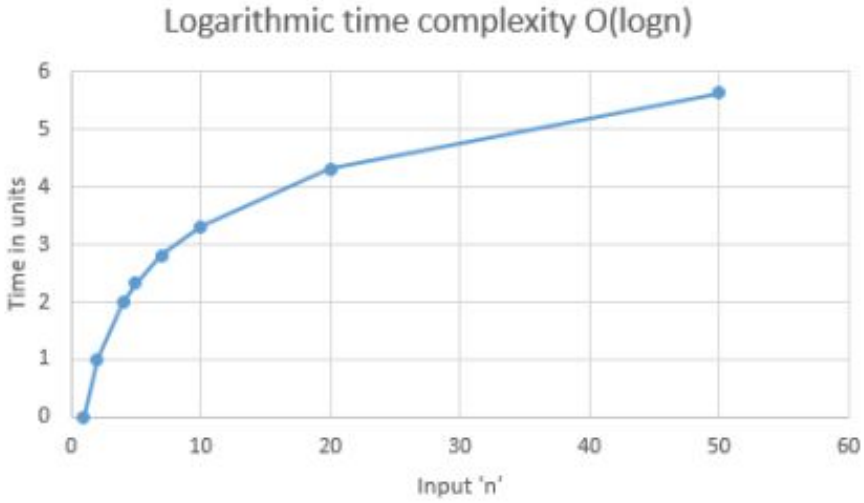


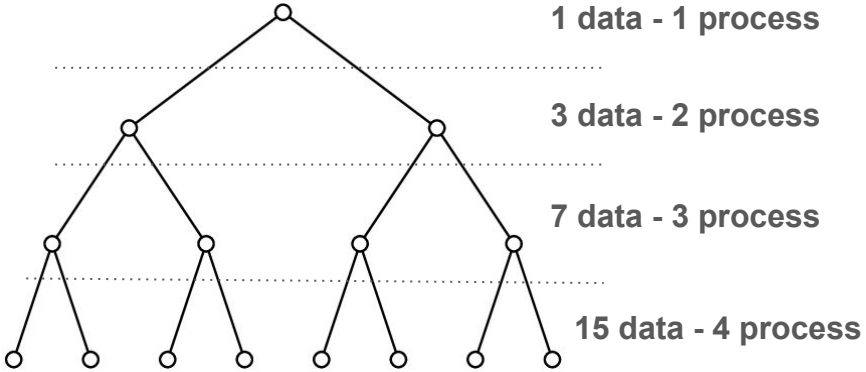
# Set example (delete 5)



# Binary Tree search and inserting

* If the tree is balanced



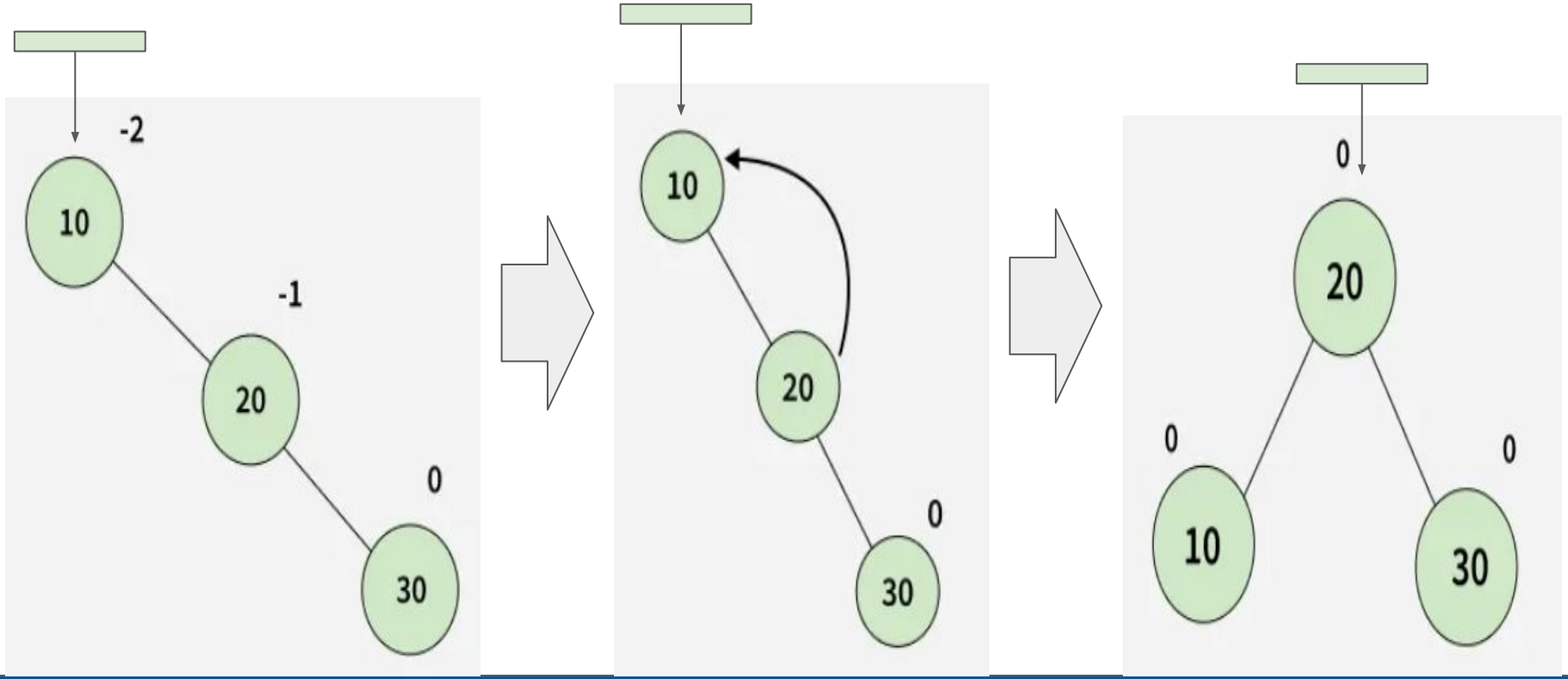
# AVL

An **AVL Tree** is a self-balancing Binary Search Tree where the **balance factor** (height of left subtree − height of right subtree) for every node is between **−1 and +1**. When this property is violated due to insertion or deletion, **rotations** are applied to restore balance.

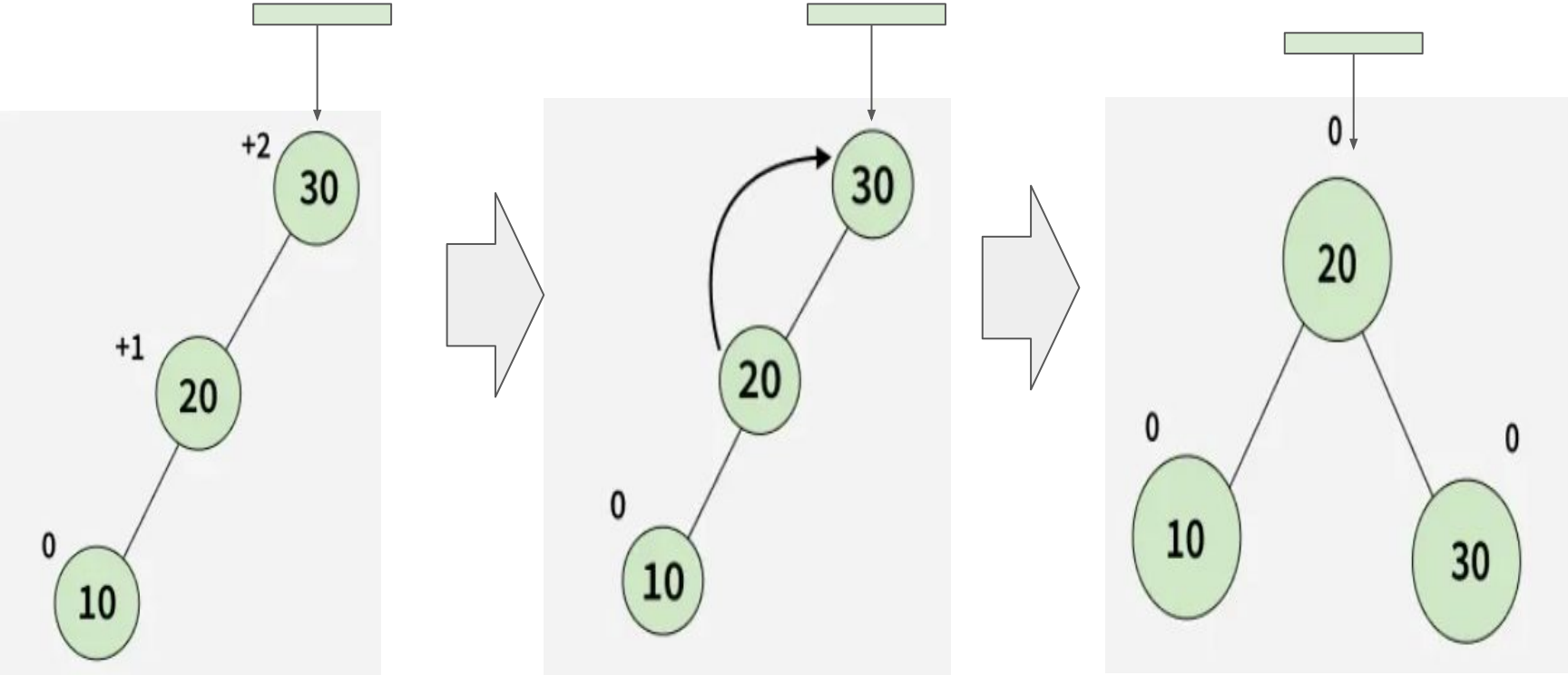
**LL (Left-Left) Rotation** occurs when a node becomes **left-heavy** because a new node is inserted into the **left subtree of its left child**. This situation is identified when:

- **Balance Factor of the node = +2**
- **Balance Factor of its left child = +1**

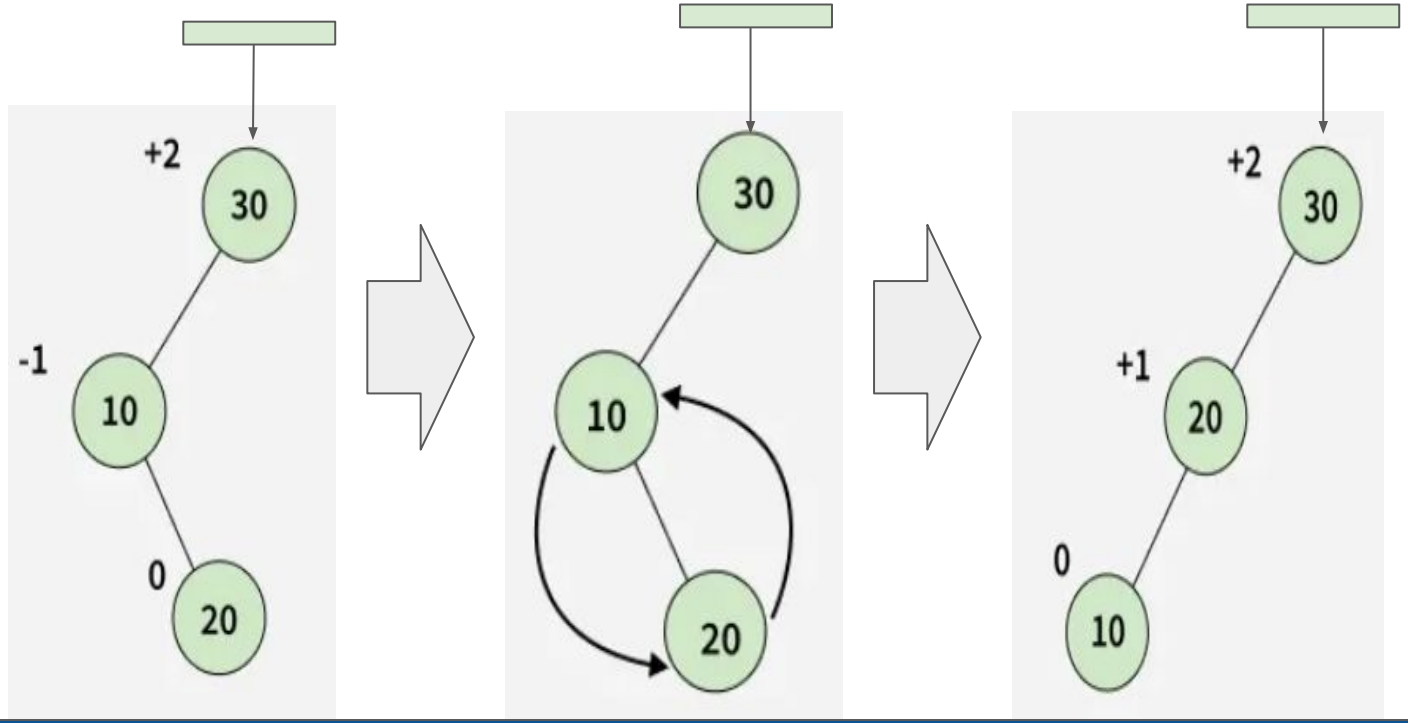
# (CASE 1) AVL - RIGHT UNBALANCED



# (CASE 2) AVL - LEFT UNBALANCED

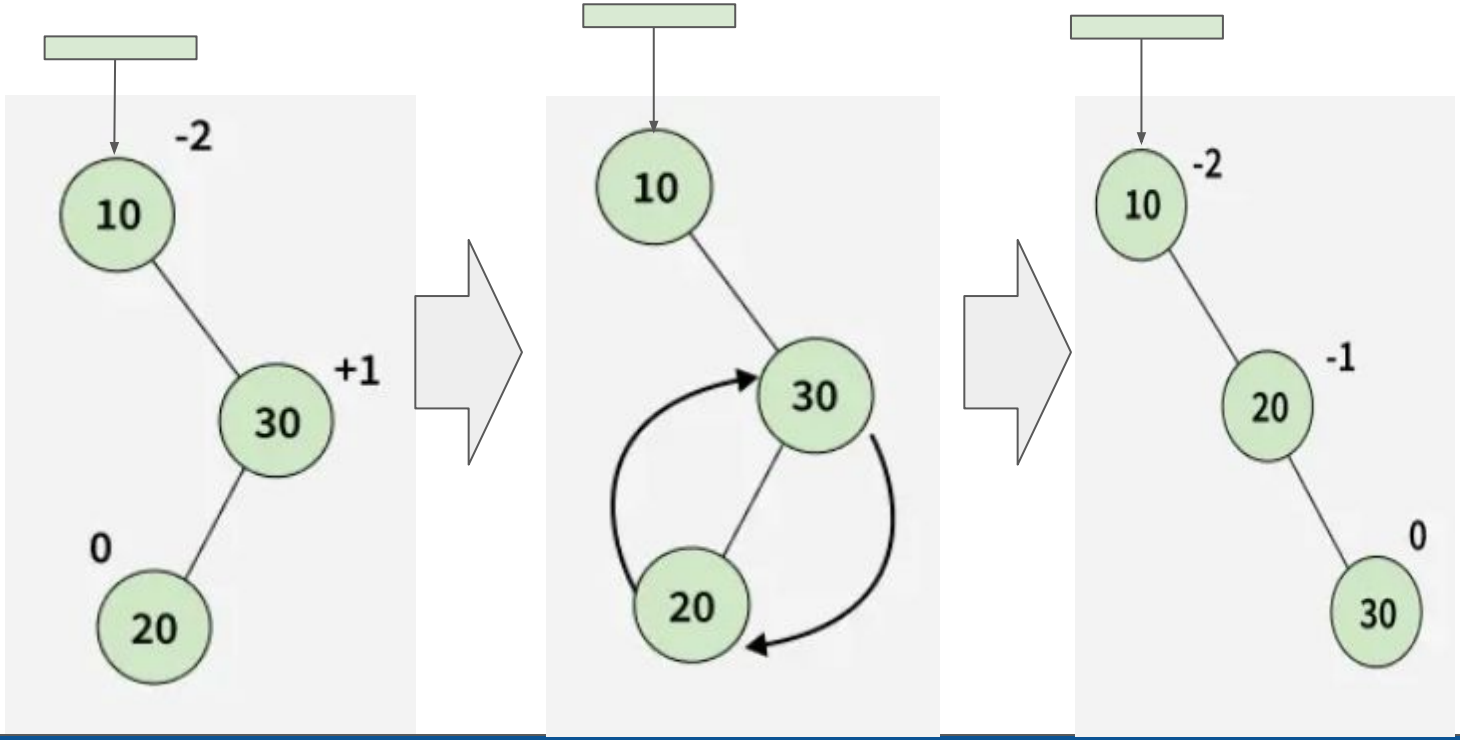


# (CASE 3) AVL - LEFT RIGHT UNBALANCED




**APPLY LEFT UNBALANCED**

# (CASE 4) AVL - RIGHT LEFT UNBALANCED




**APPLY RIGHT UNBALANCED**